Level 2
Hackathon – Fundamentals of Web Development

College Event Registration Portal

Student Details

Student Name	Raghavan R
Register Number	36725U18027
Department	II B.Sc Computer Science
ID	367
College	Global College of Arts and Science
Date of Submission	27 October 2026

Git-Hub Link: *[add your repository link here]*

1. Problem Statement

Colleges host many events every term, such as technical contests, cultural programmes, sports meets and workshops, but information about them is usually shared through notice boards, class groups and word of mouth. Students struggle to find out what is happening, to compare events by category or date, and to confirm that they actually have a seat. Organisers face the opposite problem: paper or chat-based sign-ups lead to duplicate entries, incomplete details and events that are over-booked with no easy way to track the count.

The College Event Registration Portal solves this by giving students one web page where they can explore upcoming events, read the details, register through a validated form, and manage their own registrations. The application is needed because it replaces scattered, error-prone sign-ups with a single, instant and reliable process that works on any phone or computer browser.

2. Objectives

The main objectives of the project are:

- Display all upcoming college events dynamically with title, date, venue, description and category.
- Let students search and filter events by name, category and date.
- Show detailed information for each event before the student registers.
- Provide a registration form with mandatory-field and email validation.
- Prevent duplicate registrations for the same event and enforce event capacity.
- Show the student's own registrations and allow cancellation where permitted.
- Store event and registration data with Local Storage and display registration counts for every event.

The expected outcome is a fast, one-page application that makes event discovery and registration simple for students, and gives an accurate seat count at all times, using only HTML, CSS and JavaScript.

3. Architecture

Application structure. The portal is a single-page application made of one HTML file. A static shell holds the header with live counters, a sticky tab bar (Events and My Registrations), a main container and a modal container. JavaScript keeps all data in one `state` object and re-renders the main container whenever the state changes, so no page reload is ever needed.

User navigation flow. The student lands on the Events tab, searches or filters the list, opens an event's details, and clicks Register. The form is validated; on success the registration is saved to Local Storage, counters are updated and the student can review or cancel it from the My Registrations tab.

Technologies used and their roles.

Technology	Role in the application
HTML5	Page structure: header, nav, main and a modal container.
CSS3	Styling, Flexbox/Grid layout, custom properties, sticky tab bar and media query.
JavaScript (ES6+)	Rendering, filtering, validation, event handling and storage logic.
Web Storage API	localStorage keeps registrations in the browser without a server.

Application flow diagram.

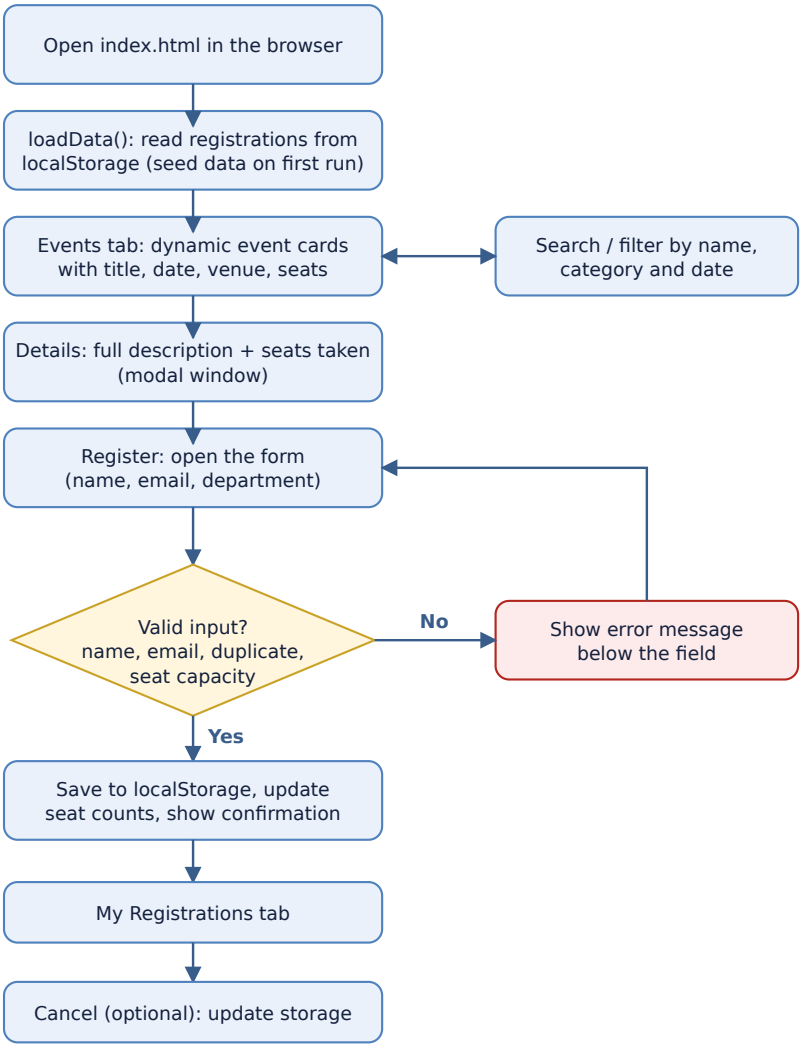


Fig. 1 – Application flow from the initial page to the final output.

4. Practical Implementation

The application was developed in four steps. First, the page structure was written in HTML5. Second, CSS custom properties, Flexbox and Grid were used to style a mobile-first layout. Third, JavaScript was added: an array of event objects feeds render functions, while one click listener and one input listener handle every interaction by event delegation. Finally, Local Storage persistence, validation rules and capacity checks were added and tested with different inputs.

4.1 Features Implemented

Feature	Purpose / description
Dynamic event display	Event cards are generated from a JavaScript array and show title, date, venue, time, category and description.
Search by name	A live search box filters the list as the student types.
Category and date filters	A category drop-down and a date picker narrow the list; all filters work together.

Event details	The Details button opens a window with the full description and seats taken.
Registration form	Collects name, email and department for the selected event.
Form validation	Name, email and department are mandatory and the email format is checked with a regular expression.
Duplicate prevention	An email address can register only once for the same event.
My Registrations	A separate tab lists only the events registered from this device, with an empty-state message.
Cancel registration	One tap removes a registration and frees the seat immediately.
Local Storage persistence	All registrations are saved in localStorage, so they survive page reloads.
Registration counts	Every card shows "registered / capacity"; the header shows total seats left.
Capacity validation	Full events show "Event full", disable Register and are re-checked on submit.
Responsive design	A fluid single-column layout with a bottom-sheet form on phones that centres on wide screens.

4.2 Source Code

The important sections of the source code are shown below. The complete file (index.html) is submitted along with this report.

HTML

HTML provides the page skeleton: a header with the college name and live counters, a navigation bar with two tabs, a `main` element into which every view is rendered, and an empty `div` used for the details and registration windows.

```
<header>
  <h1>College Event Registration Portal</h1>
  <p>Global College of Arts and Science</p>
  <div class="stats">
    <div><b id="sEvents">0</b>Events</div>
    <div><b id="sMine">0</b>My registrations</div>
    <div><b id="sSeats">0</b>Seats left</div>
  </div>
</header>
<nav id="tabs">
  <button data-tab="events" class="active">Events</button>
  <button data-tab="mine">My Registrations</button>
</nav>
<main id="main"></main>
<div id="modal"></div>
```

CSS

CSS styles the event cards, tags, buttons (including the disabled state for full events) and the pop-up sheet. Custom properties define the colour palette once, Flexbox aligns the card rows, and a media query centres the sheet on wide screens.

```
.card{background:#fff;border:1px solid var(--line);border-radius:10px;padding:14px;margin-
bottom:12px}
.card h3{margin:0 0 4px;font-size:16px}
.meta{font-size:13px;color:var(--muted)}
.tag{display:inline-block;background:#e6efff;color:var(--blue);border-radius:10px;
  padding:2px 9px;font-size:12px;margin-bottom:6px}
.row{display:flex;justify-content:space-between;align-items:center;margin-top:10px;gap:8px}
.count{font-size:13px}
.count.full{color:var(--bad);font-weight:600}
.btn:disabled{background:#c7ced9;color:#fff;border:0;cursor:not-allowed}
.modal{position:fixed;inset:0;background:rgba(0,0,0,.5);display:flex;
  align-items:flex-end;justify-content:center;z-index:10}
.sheet{background:#fff;width:100%;max-width:760px;max-height:92vh;overflow:auto;
  border-radius:16px 16px 0 0;padding:18px}
@media(min-width:600px){.modal{align-items:center}.sheet{border-radius:16px}}
```

JavaScript

JavaScript holds the application logic. `loadData()` restores registrations from Local Storage (or seeds starter data). `submitReg()` validates the form, checks for a duplicate email and a full event, saves the new registration and refreshes the page. `cancel()` removes a registration and updates the counters.

```
function loadData(){
  try { const r = localStorage.getItem('regs'); if (r) { state.regs = JSON.parse(r); return; } }
catch(e) {}
  let n = 0;
  Object.entries(SEED).forEach(([id, k]) => { for (let i = 0; i < k; i++) { n++;
    state.regs.push({id:'s'+n, eventId:+id, name:'Student '+n, email:'student'+n+'@college.edu',
mine:false}); } });
  save();
}
const count = id => state.regs.filter(r => r.eventId === id).length;
const mine = () => state.regs.filter(r => r.mine);
function submitReg(id){
  const e = EVENTS.find(x => x.id === id);
  const name = $('#fn').value.trim(), email = $('#em').value.trim().toLowerCase(), dept =
$('#dp').value.trim();
  const errs = {
    e1: name ? '' : 'Name is required',
    e2: !email ? 'Email is required'
      : !/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/.test(email) ? 'Enter a valid email address'
      : state.regs.some(r => r.eventId === id && r.email === email) ? 'This email is already
registered for this event' : '',
    e3: dept ? '' : 'Department is required'
  };
  Object.entries(errs).forEach(([k, v]) => { $('#' + k).textContent = v; });
  if (Object.values(errs).some(Boolean)) return;
  if (count(id) >= e.capacity) { closeModal(); toast('Sorry, this event is now full'); return
refresh(); }
  state.regs.push({id:'r' + Date.now(), eventId:id, name, email, dept, mine:true});
  save(); closeModal(); toast('Registered for ' + e.title); refresh();
}
function cancel(rid){
  state.regs = state.regs.filter(r => r.id !== rid);
  save(); toast('Registration cancelled'); refresh();
}
```

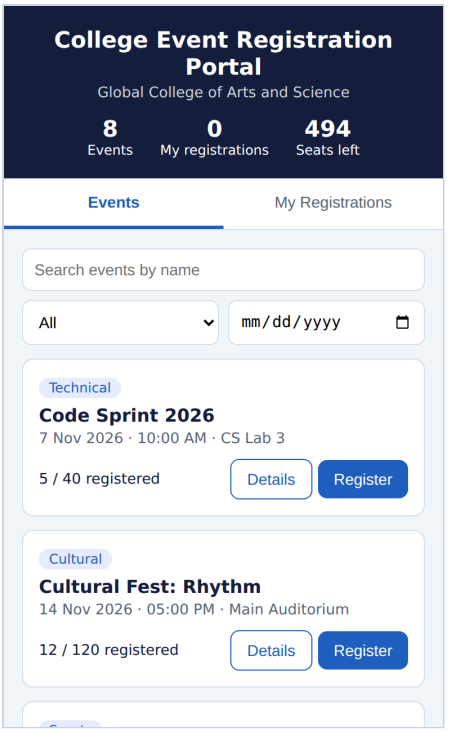
4.3 Project Execution

To run: download the project folder and open `index.html` in any modern browser (Chrome, Edge, Firefox or Safari). No server, installation or internet connection is required.

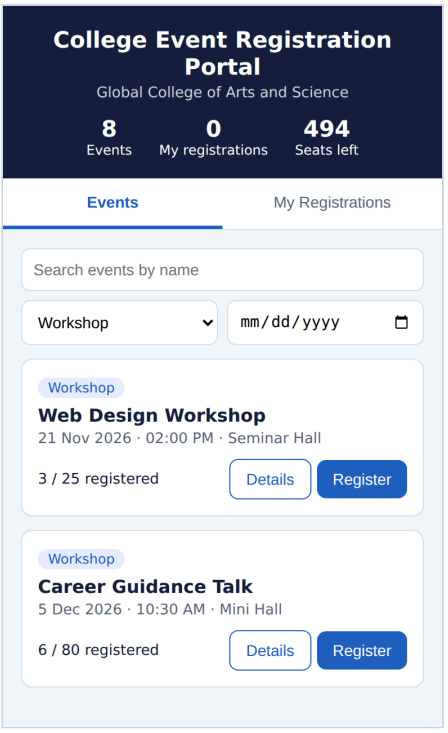
Working process: when the page opens, `loadData()` reads Local Storage and the event cards appear with their seat counts. The student types in the search box or picks a category or date and the list updates instantly. Tapping Details shows the full information; tapping Register opens the form. After the student enters name, email and department and taps Submit, the input is validated. Errors are shown beside the fields; if everything is valid the registration is saved, the seat count and header counters increase, and a confirmation message appears. The My Registrations tab then lists the student's events, where any registration can be cancelled.

5. Output Screenshots

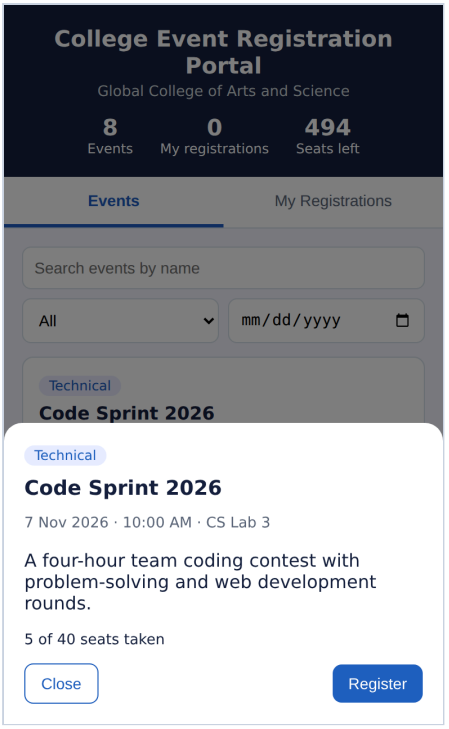
The screenshots below were taken from the working application.



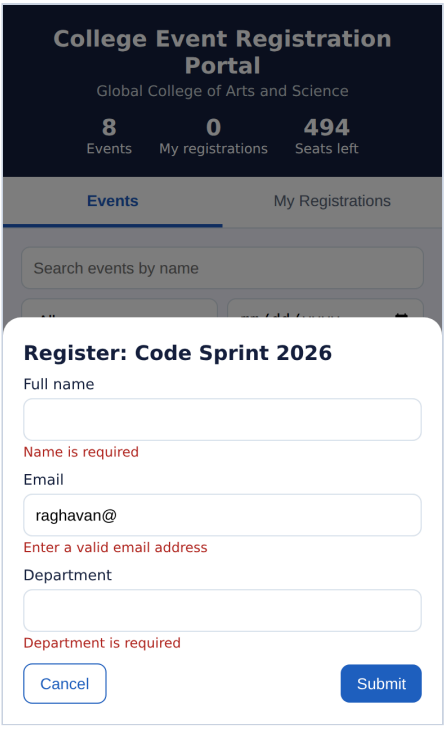
Screenshot 1 – Event Listing
Home page with live counters and event cards generated from the data array.



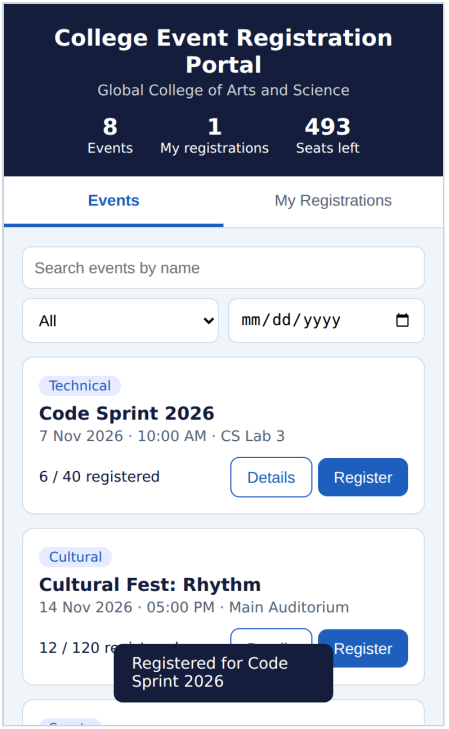
Screenshot 2 – Search and Filter
Category filter applied; search and date filters update the list instantly.



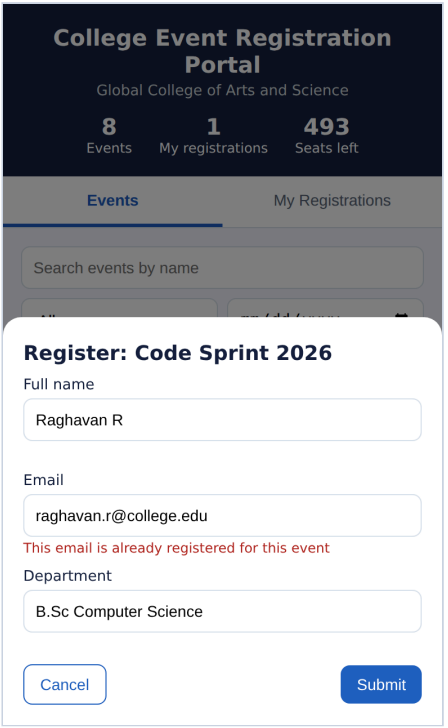
Screenshot 3 – Event Details
Detail window showing description, venue, time and seats taken.



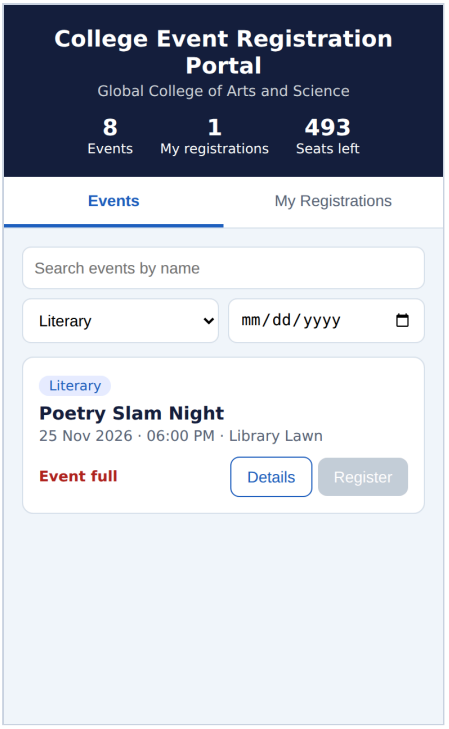
Screenshot 4 – Form Validation
Mandatory-field and email validation messages on the registration form.



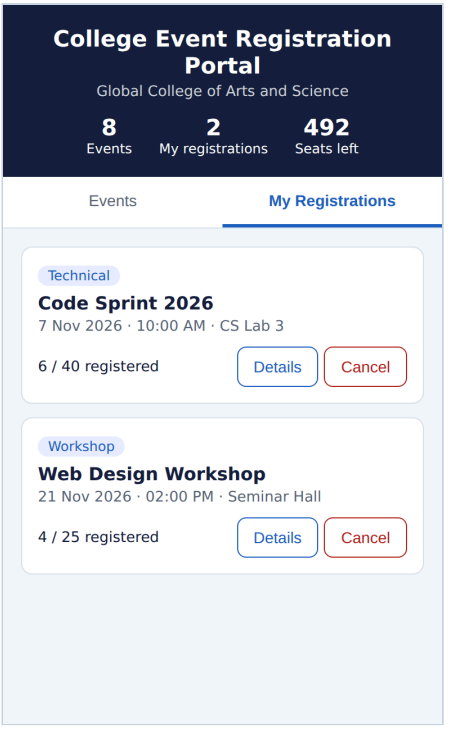
Screenshot 5 – Successful Registration
Confirmation message; the seat count and header counters update at once.



Screenshot 6 – Duplicate Prevention
The same email cannot register twice for the same event.



Screenshot 7 – Capacity Validation
A full event shows "Event full" and its Register button is disabled.



Screenshot 8 – My Registrations
The student's own registrations with a Cancel button for each.

6. Technical Explanation

HTML5. Semantic elements (`header`, `nav`, `main`) structure a single static shell. Form fields use `label` elements linked with `for` and `id`, and the viewport meta tag makes the page scale correctly on phones.

CSS3. Custom properties (`--ink`, `--blue`, `--bad`) hold the palette. Grid lays out the filter controls, Flexbox aligns card rows and the tab bar, `position: sticky` keeps the tabs visible, `position: fixed` creates the modal overlay, and a media query adapts the layout for wide screens.

JavaScript concepts. The program is built around a single state object and pure render functions that return template-literal HTML. Event delegation attaches only two listeners to the document, so newly rendered buttons work without re-binding. Array methods drive the logic: `filter()` for search and filters, `sort()` for date order, `some()` for duplicate detection and `reduce()` for total seats left. A regular expression validates email addresses, and user text is escaped before being inserted into HTML to prevent script injection. Local Storage access is wrapped in `try/catch` so a storage failure never breaks the page.

Key functions.

Function	Responsibility
loadData()	Reads saved registrations from localStorage, or seeds starter data on the first run.
render() / listHTML() / card()	Turn the current state into HTML for the active tab and the filtered event list.

detail(id) / form(id)	Open the event details window and the registration form.
submitReg(id)	Validates the form, blocks duplicates and full events, then saves the registration.
cancel(id)	Removes a registration, saves, and refreshes counters.
count(id) / mine()	Helpers that return seats taken for an event and the student's own registrations.

7. Conclusion

The College Event Registration Portal meets the stated requirements: events are displayed dynamically, can be searched and filtered, show detailed information, and accept validated registrations that are saved in Local Storage. Duplicate and over-capacity registrations are blocked, students can view and cancel their registrations, and every event shows its registration count. Building the project strengthened my understanding of semantic HTML, responsive CSS, DOM manipulation, event delegation, form validation and the Web Storage API, and showed how much can be achieved without a framework.

8. Future Enhancements

- Add a back-end and database so registrations are shared across devices and organisers see live totals.
- Add secure student login and an admin panel to create, edit and delete events.
- Send confirmation emails and reminders, and issue a QR-code entry pass for each registration.
- Add a waiting list that automatically promotes students when a registration is cancelled.
- Offer calendar export (.ics), event photos, and a downloadable attendance report for organisers.
- Improve accessibility with keyboard focus management, screen-reader labels and a dark mode.